

DEVOPS · BACK-END · LINUX

Deploy de Aplicação Python em Produção

Flask + Gunicorn + Nginx + Systemd no Ubuntu — guia completo de estudo com explicação detalhada de cada camada e decisão de arquitetura.

Python / Flask

Gunicorn

Nginx

Systemd

Ubuntu

Autor: Willians Martins · **Foco:** Desenvolvimento com visão de segurança

Repositório: github.com/Willsmt/flask-gunicorn-nginx-deploy

Sumário

1. Visão Geral — o problema que esse stack resolve

2. Arquitetura completa

3. Ambiente Virtual (venv)

4. A aplicação Flask

5. O entry point WSGI

6. Gunicorn — servidor de produção

7. Serviço Systemd

8. Unix Socket vs Porta TCP (visão de segurança)

9. Nginx como Reverse Proxy

10. DNS local e teste final

11. Troubleshooting — erros enfrentados

12. Comandos de referência rápida

13. Glossário de termos

14. Próximos passos (SSL, Django, CI/CD)

1. Visão Geral

Quando você roda `python app.py`, o Flask sobe um servidor **de desenvolvimento**. Ele resolve o problema imediato — "ver a aplicação rodando" — mas esconde uma pergunta crítica: *quem mantém a aplicação viva quando algo dá errado?*

O servidor embutido do Flask tem limitações que o tornam inadequado para produção:

Limitação	Consequência em produção
Single-thread	Atende uma requisição por vez — trava sob carga
Sem gestão de falhas	Se a app travar, fica fora do ar até intervenção manual
Sem integração com o SO	Não sobe no boot, morre ao fechar o terminal
Sem camada de proteção	Exposto diretamente, sem proxy, SSL ou rate limiting

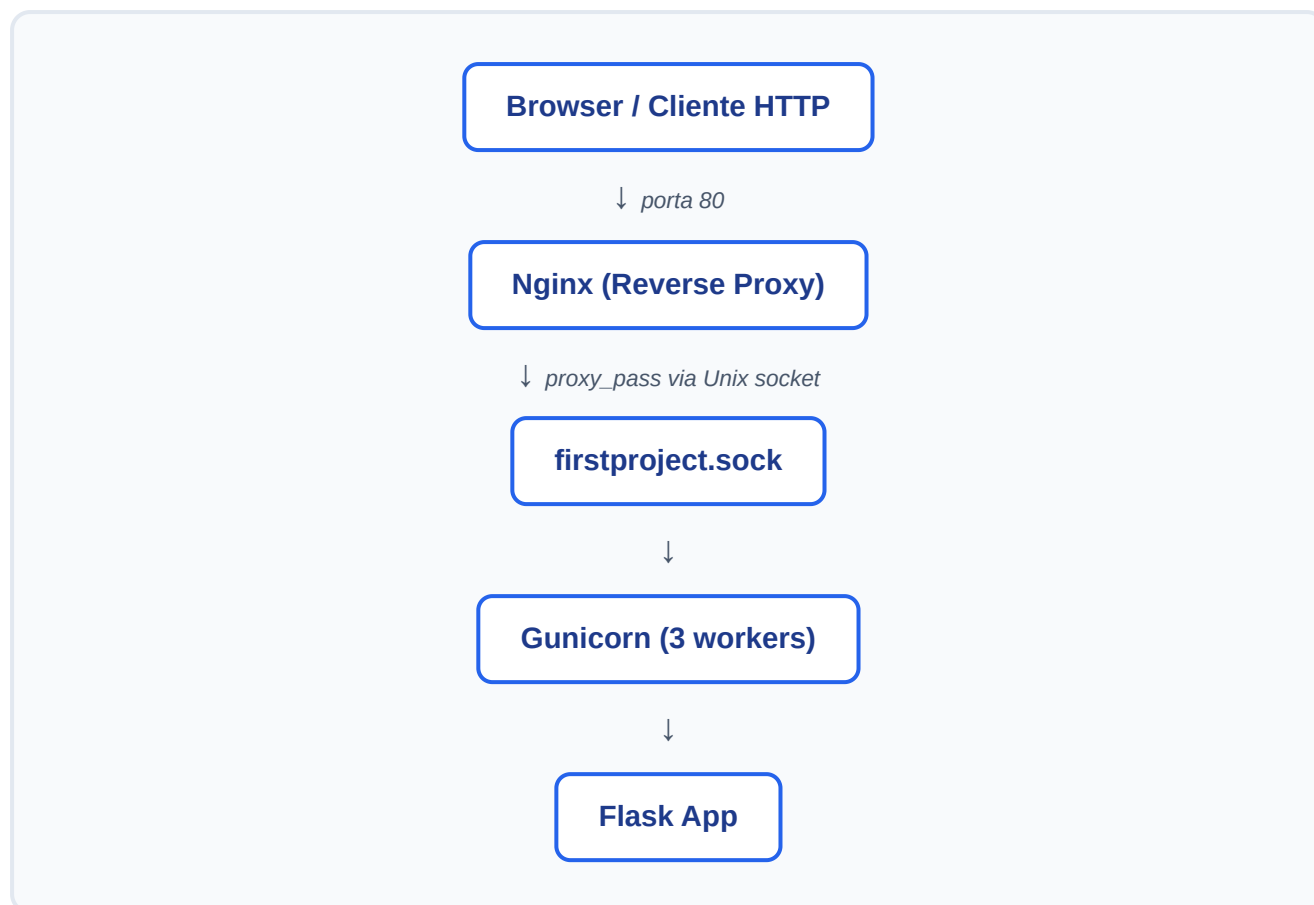
A solução é uma arquitetura de **três camadas**, cada uma com uma responsabilidade específica. Esse é o padrão usado em ambientes de produção reais.

Por que aprender isso importa

Entender o porquê de cada camada — e não só executar comandos — é o que diferencia quem "fez funcionar" de quem "entende como funciona". Cada decisão aqui tem um motivo de performance, confiabilidade ou segurança.

2. Arquitetura Completa

O fluxo de uma requisição percorre todas as camadas até a aplicação e volta pelo mesmo caminho:



Responsabilidade de cada camada

Camada	Tecnologia	Responsabilidade
Isolamento	Python venv	Dependências isoladas por projeto
Aplicação	Flask	Lógica da aplicação web
Entry point	wsgi.py	Interface padrão WSGI
Servidor WSGI	Gunicorn	Processa Python, multi-worker
Gerenciador	Systemd	Mantém o processo vivo
Reverse proxy	Nginx	Único ponto de entrada HTTP

3. Ambiente Virtual (venv)

O problema sem isolamento

Quando você instala um pacote com `pip install flask` sem isolamento, ele vai para o Python **global** do sistema. Imagine dois projetos: o Projeto A precisa de `flask==2.0` e o Projeto B precisa de `flask==3.1`. É impossível manter os dois instalados globalmente — um quebra o outro.

A solução — um Python isolado por projeto

O ambiente virtual cria uma cópia isolada do Python dentro da pasta do projeto. Cada projeto tem suas próprias dependências, sem interferência.

```
# Criar o ambiente virtual (Python 3.3+ já vem com venv)
python3 -m venv firstprojectenv

# Ativar — o terminal passa a exibir (firstprojectenv)
source firstprojectenv/bin/activate

# Instalar dependências DENTRO do venv (sem sudo)
pip install gunicorn flask

# Sair do ambiente
deactivate
```

venv vs virtualenv

O `venv` é embutido no Python 3 e é o padrão atual. O `virtualenv` é um pacote externo, usado antes do Python 3.3. O comportamento é idêntico — use `venv`.

PEP 668 — externally-managed-environment

O Ubuntu moderno bloqueia `pip install` global para proteger pacotes gerenciados pelo `apt`. Esse é o motivo de sempre trabalhar dentro do `venv`. A flag `--break-system-packages` força o override, mas deve ser evitada.

Por que o Gunicorn precisa estar DENTRO do venv

O systemd aponta para o executável do Gunicorn dentro do venv. Se o Gunicorn estiver instalado globalmente, ele não enxerga o Flask nem os pacotes do venv — a aplicação quebra na inicialização. Sempre verifique com `which gunicorn` que o caminho aponta para dentro do venv.

4. A Aplicação Flask

Arquivo `firstproject.py` — a aplicação mínima:

```
from flask import Flask

application = Flask(__name__)

@application.route("/")
def hello():
    return "<h1 style='color:blue'>Hello There!</h1>"

if __name__ == "__main__":
    application.run(host='0.0.0.0')
```

Explicação linha a linha

Linha	O que faz
<code>from flask import Flask</code>	Importa a classe Flask do pacote
<code>application = Flask(__name__)</code>	Cria a instância. <code>__name__</code> diz ao Flask onde está o arquivo raiz
<code>@application.route("/")</code>	Decorator que mapeia a URL <code>/</code> para a função abaixo
<code>def hello()</code>	Função executada quando a rota é acessada
<code>return "<h1...>"</code>	Retorna HTML como resposta HTTP
<code>application.run(host='0.0.0.0')</code>	Sobe o servidor em todas as interfaces de rede

Por que `application` e não `app` ?

O Gunicorn e o protocolo WSGI procuram por uma variável chamada `application` por convenção. Se você usar `app`, precisaria informar explicitamente o nome no comando do Gunicorn (`wsgi:app`).

O que é `host='0.0.0.0'` ?

Por padrão o Flask escuta apenas em `127.0.0.1` (localhost) — acessível só pela própria máquina. O `0.0.0.0` faz escutar em todas as interfaces de rede, permitindo acesso de outras máquinas na rede local (essencial quando você acessa via SSH de outro computador).

5. O Entry Point WSGI

Arquivo `wsgi.py` :

```
from firstproject import application

if __name__ == "__main__":
    application.run()
```

O que é WSGI?

WSGI (Web Server Gateway Interface) é o **protocolo padrão** que conecta servidores web (Gunicorn, uWSGI) com aplicações Python. O Gunicorn não sabe falar diretamente com o Flask — ele fala com o protocolo WSGI. Esse arquivo é o ponto de entrada que respeita esse protocolo.

Por que esse arquivo existe

Sem o `wsgi.py`, você teria que passar o caminho completo do módulo toda vez que subisse o Gunicorn. Com ele, o comando fica limpo: `gunicorn wsgi` — o Gunicorn importa o módulo `wsgi.py` e pega a variável `application` de dentro.

Teste via Gunicorn

```
gunicorn --bind 0.0.0.0:8000 wsgi
```

Acessando `http://IP:8000` a aplicação responde — agora servida pelo Gunicorn, não mais pelo servidor de desenvolvimento do Flask.

6. Gunicorn — Servidor de Produção

O Gunicorn (Green Unicorn) é um servidor WSGI projetado para produção. Diferente do servidor embutido do Flask, ele:

- **Cria múltiplos workers** — processos paralelos que atendem várias requisições simultaneamente
- **Gerencia crashes** — se um worker morre, o Gunicorn sobe outro automaticamente
- **É performático** — escrito para lidar com carga real de produção

Anatomia do comando

```
gunicorn --workers 3 --bind unix:firstproject.sock -m 007 wsgi
```

Parâmetro	Função
<code>--workers 3</code>	3 processos paralelos. Regra geral: $(2 \times \text{núcleos da CPU}) + 1$
<code>--bind unix:...sock</code>	Cria um Unix socket em vez de abrir uma porta TCP
<code>-m 007</code>	Define a máscara de permissão do socket
<code>wsgi</code>	Módulo de entrada (arquivo <code>wsgi.py</code>)

Worker timeout ao acessar direto

Ao acessar o Gunicorn diretamente pelo browser (sem Nginx), pode aparecer `WORKER TIMEOUT` no log. Isso ocorre porque o browser mantém conexões keep-alive abertas que o Gunicorn não gerencia bem sozinho. É exatamente uma das razões pelas quais o Nginx existe nesse stack — ele gerencia essas conexões antes de repassar ao Gunicorn. O erro desaparece com o Nginx na frente.

7. Serviço Systemd

Sem o systemd, o Gunicorn morre quando você fecha o terminal. O systemd é o gerenciador de processos do Linux — ele mantém a aplicação rodando em background, reinicia se cair e sobe no boot.

Arquivo `/etc/systemd/system/firstproject.service` :

```
[Unit]
Description=Gunicorn instance to serve firstproject
After=network.target

[Service]
User=willians
Group=willians
WorkingDirectory=/home/willians/projects/firstproject
Environment="PATH=/home/willians/projects/firstproject/firstprojectenv/bin"
ExecStart=/home/willians/projects/firstproject/firstprojectenv/bin/gunicorn \
    --workers 3 \
    --bind unix:firstproject.sock \
    -m 007 \
    wsgi

[Install]
WantedBy=multi-user.target
```

Explicação dos blocos

[Unit] — metadados e ordem de inicialização

Diretiva	Função
<code>Description</code>	Nome legível do serviço nos logs
<code>After=network.target</code>	Só inicia após a rede estar disponível

[Service] — como o processo roda

Diretiva	Função
<code>User</code> / <code>Group</code>	Roda como seu usuário, não como root (princípio do menor privilégio)
<code>WorkingDirectory</code>	Pasta onde o Gunicorn executa
<code>Environment="PATH=..."</code>	Aponta para o Python/Gunicorn do venv
<code>ExecStart</code>	Comando exato que sobe o Gunicorn

[Install] — comportamento no boot

Diretiva	Função
<code>WantedBy=multi-user.target</code>	Sobe junto com o sistema no boot multiusuário

Ativando o serviço

```
# Recarrega o systemd para reconhecer o novo arquivo
sudo systemctl daemon-reload

# Inicia o serviço agora
sudo systemctl start firstproject

# Habilita para subir no boot
sudo systemctl enable firstproject

# Verifica o status
sudo systemctl status firstproject
```

Princípio do menor privilégio

Rodar o serviço como `User=willians` em vez de root significa que, se a aplicação for comprometida, o atacante tem apenas os privilégios do seu usuário — não controle total do sistema. É uma prática fundamental de segurança defensiva.

8. Unix Socket vs Porta TCP

Essa é a decisão de arquitetura mais importante do ponto de vista de segurança. Em vez de o Gunicorn abrir uma porta TCP (`0.0.0.0:8000`), ele cria um arquivo de socket Unix.

Com porta TCP — superfície de ataque maior



Se o atacante descobrir a porta 8000 (um `nmap` resolve em segundos), ele **pula o Nginx completamente**. Sem rate limiting, sem logs do Nginx, sem headers de segurança. Direto na aplicação Python.

Com Unix Socket — superfície mínima



O socket é um **arquivo no sistema de arquivos**. Não tem porta, não aparece no `nmap`, não é acessível pela rede. Apenas processos na mesma máquina com permissão de leitura conseguem se comunicar com ele.

A permissão `-m 007`

O socket criado tem permissão `srwxrwx---`:

Dono (willians)	Grupo (willians)	Outros
rwx (total)	rwx (total)	--- (nenhum)

O `s` inicial indica que é um socket file. Apenas o dono e o grupo acessam — qualquer outro processo do sistema é bloqueado.

Visão Blue Team

Reduzir a superfície de ataque é um princípio central de segurança defensiva. Cada porta aberta é um vetor potencial. Ao usar socket, o único ponto de entrada é o Nginx — que pode ser endurecido com rate limiting, WAF, headers de segurança e SSL.

9. Nginx como Reverse Proxy

O Nginx fica na frente de tudo. Ele recebe as requisições HTTP, serve arquivos estáticos diretamente e repassa as requisições dinâmicas para o Gunicorn via socket.

Arquivo `/etc/nginx/sites-available/firstproject` :

```
server {
    listen 80;
    server_name firstproject www.firstproject;

    access_log /var/log/nginx/firstproject.access.log;
    error_log /var/log/nginx/firstproject.error.log;

    location / {
        include proxy_params;
        proxy_pass http://unix:/home/willians/projects/firstproject/firstproject.sock;
    }
}
```

Explicação das diretivas

Diretiva	Função
<code>listen 80</code>	Escuta na porta padrão do HTTP
<code>server_name</code>	Quais domínios esse bloco responde — permite hospedar vários sites
<code>access_log / error_log</code>	Logs separados por projeto — facilita análise de incidentes
<code>location /</code>	Comportamento para todas as rotas
<code>include proxy_params</code>	Injeta headers (IP real, protocolo) antes de repassar
<code>proxy_pass</code> <code>http://unix:...</code>	Repassa a requisição para o socket do Gunicorn

Por que `proxy_params` é importante

O Gunicorn recebe as requisições do Nginx, não do browser. Sem esses headers (`X-Real-IP` , `X-Forwarded-For`), o Gunicorn pensaria que todas vêm do localhost. Com eles, a aplicação sabe o IP real do visitante — essencial para logs, auditoria e segurança.

sites-available vs sites-enabled

O Nginx usa dois diretórios por convenção:

Diretório	Função
<code>sites-available/</code>	Todos os sites configurados (ativos ou não)
<code>sites-enabled/</code>	Symlinks (atalhos) para os sites ativos

Para ativar, cria-se um symlink — assim é possível ativar/desativar sites sem apagar a configuração:

```
# Ativar o site
sudo ln -s /etc/nginx/sites-available/firstproject /etc/nginx/sites-enabled/

# Testar a sintaxe da configuração
sudo nginx -t

# Reiniciar o Nginx
sudo systemctl restart nginx
```

10. DNS Local e Teste Final

Sem um DNS real, o sistema não sabe que `http://firstproject` aponta para sua máquina. O arquivo `/etc/hosts` funciona como um DNS local.

```
# Adicionar ao final de /etc/hosts
192.168.0.106    firstproject www.firstproject
```

Domínio local vs domínio real

O `/etc/hosts` só funciona na sua máquina e rede local — é um domínio simulado, gratuito, ideal para desenvolvimento. Um domínio real (comprado em um registrar) é resolvido por servidores DNS globais e acessível por qualquer pessoa. Em produção, só muda o `server_name` no Nginx — o resto permanece igual.

Teste com curl

```
curl firstproject
```

Deve retornar o HTML da aplicação: `<h1 style='color:blue'>Hello There!</h1>`

Acessando de outra máquina (Windows via SSH)

Quando você acessa o servidor Ubuntu via SSH a partir do Windows, o `localhost` do browser aponta para o Windows, não para o Ubuntu. Para resolver o domínio no Windows, adicione a mesma linha em:

```
C:\Windows\System32\drivers\etc\hosts
```

Editado com o Bloco de Notas executado como Administrador.

11. Troubleshooting — Erros Enfrentados

Erro: externally-managed-environment

Sintoma

`error: externally-managed-environment` ao rodar `pip install` globalmente.

Causa: PEP 668 — o Ubuntu protege o Python do sistema.

Solução: trabalhar dentro do venv (`source venv/bin/activate`) ou usar `sudo apt install python3-venv` .

Erro: 502 Bad Gateway

Sintoma

Nginx retorna `502 Bad Gateway` ao acessar o domínio.

Causa: o Nginx (usuário `www-data`) não tem permissão para acessar o socket criado pelo usuário `willians` . Pela máscara `007` , "outros" tem permissão zero.

Solução: adicionar `www-data` ao grupo do usuário:

```
sudo usermod -aG willians www-data
sudo systemctl restart firstproject
sudo systemctl restart nginx
```

Erro: Welcome to nginx (página padrão)

Sintoma

Acessar pelo IP mostra a página padrão do Nginx em vez da aplicação.

Causa: a requisição via IP não bate com o `server_name firstproject` , então cai no site default.

Solução: acessar pelo domínio (adicionar ao hosts do cliente) ou desativar o site default com `sudo rm /etc/nginx/sites-enabled/default` .

Erro: WORKER TIMEOUT

Sintoma

`WORKER TIMEOUT` nos logs ao acessar o Gunicorn diretamente.

Causa: conexões keep-alive do browser que o Gunicorn não gerencia bem sozinho.

Solução: colocar o Nginx na frente — ele gerencia as conexões antes de repassar. O erro desaparece com a arquitetura completa.

Erro: gunicorn instalado globalmente

Sintoma

`which gunicorn` retorna `~/.local/bin/gunicorn` em vez do caminho do venv.

Causa: instalação anterior com `pip install --user`.

Solução: remover os arquivos manualmente e reinstalar dentro do venv:

```
find ~/.local -name "*gunicorn*"
rm ~/.local/bin/gunicorn ~/.local/bin/gunicornc
rm -rf ~/.local/lib/python3.x/site-packages/gunicorn*
```

12. Comandos de Referência Rápida

Ambiente Virtual

```
python3 -m venv venv          # criar
source venv/bin/activate      # ativar
deactivate                    # sair
pip install pacote            # instalar (dentro do venv)
pip freeze > requirements.txt # exportar dependências
```

Systemd

```
sudo systemctl daemon-reload # recarregar configs
sudo systemctl start firstproject # iniciar
sudo systemctl stop firstproject # parar
sudo systemctl restart firstproject # reiniciar
sudo systemctl enable firstproject # habilitar no boot
sudo systemctl status firstproject # ver status
sudo journalctl -u firstproject -f # ver logs em tempo real
```

Nginx

```
sudo nginx -t          # testar sintaxe
sudo systemctl restart nginx # reiniciar
sudo systemctl reload nginx # recarregar sem derrubar
sudo tail -f /var/log/nginx/firstproject.access.log # logs de acesso
sudo tail -f /var/log/nginx/firstproject.error.log # logs de erro
```

Gunicorn

```
gunicorn --bind 0.0.0.0:8000 wsgi # teste rápido
gunicorn --workers 3 --bind unix:app.sock -m 007 wsgi # produção
```

Git via SSH

```
ssh-keygen -t ed25519 -C "email"      # gerar chave
cat ~/.ssh/id_ed25519.pub             # exibir chave pública
ssh -T git@github.com                 # testar conexão
git init                             # iniciar repo
git add arquivo                       # adicionar ao staging
git commit -m "feat(escopo): msg"     # commit
git remote add origin git@github.com:user/repo.git
git push -u origin main                # enviar
```

13. Glossário de Termos

Termo	Definição
WSGI	Web Server Gateway Interface. Protocolo padrão entre servidores web e aplicações Python.
venv	Virtual Environment. Ambiente Python isolado por projeto.
Gunicorn	Green Unicorn. Servidor WSGI de produção com múltiplos workers.
Worker	Processo paralelo do Gunicorn que atende requisições.
Systemd	Gerenciador de serviços e processos do Linux moderno.
Reverse Proxy	Servidor que recebe requisições e as repassa para um backend interno.
Unix Socket	Arquivo de comunicação entre processos na mesma máquina, sem usar rede.
TCP Port	Ponto de comunicação de rede identificado por número, acessível remotamente.
PEP 668	Especificação que protege o Python do sistema de instalações pip globais.
symlink	Symbolic link. Atalho que aponta para outro arquivo ou diretório.
/etc/hosts	Arquivo de DNS local que mapeia nomes para IPs na própria máquina.
502 Bad Gateway	Erro HTTP indicando que o proxy não conseguiu se comunicar com o backend.
keep-alive	Conexão HTTP persistente, reutilizada para múltiplas requisições.
www-data	Usuário sob o qual o Nginx roda por padrão no Ubuntu.
Superfície de ataque	Conjunto de pontos por onde um sistema pode ser atacado.
Menor privilégio	Princípio de segurança: dar a cada processo apenas as permissões necessárias.

14. Próximos Passos

HTTPS com Certbot + Let's Encrypt

Atualmente o browser exibe "Não seguro" porque o tráfego é HTTP puro. O próximo passo é emitir um certificado SSL gratuito e configurar HTTPS:

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d firstproject
```

O Certbot configura automaticamente o Nginx para HTTPS e renova o certificado periodicamente. **Observação:** requer um domínio real — não funciona com domínio local do `/etc/hosts`.

Por que HTTPS importa

Sem SSL, todo o tráfego (incluindo senhas e dados sensíveis) trafega em texto puro, vulnerável a interceptação. HTTPS criptografa a comunicação e garante a identidade do servidor. É requisito básico de segurança para qualquer aplicação real.

Migrar de Flask para Django

O mesmo stack funciona com Django — apenas o `wsgi.py` muda (o Django já gera o seu automaticamente em `projeto/wsgi.py`). O Gunicorn, systemd e Nginx permanecem idênticos.

Variáveis de ambiente

Para projetos com banco de dados e chaves de API, use `python-decouple` ou `python-dotenv` com um arquivo `.env` — nunca versionar credenciais no Git.

CI/CD com GitHub Actions

Automatizar o deploy: a cada `git push` na branch main, um workflow conecta via SSH ao servidor, atualiza o código e reinicia o serviço systemd.

Conclusão

Esse projeto demonstra que colocar uma aplicação em produção vai muito além de escrever código. Cada camada — venv, Gunicorn, systemd, Nginx, socket — existe por um motivo de confiabilidade, performance ou segurança. Entender o porquê de cada decisão é o que transforma um desenvolvedor em um profissional com visão de arquitetura e segurança.